Gate 07 — [MVP GATE] Public Frontend

> ***SUPERSEDED 2026-04-17.*** *This is the pre-architecture spec, retained for history. Planning of record is* `gate-07-mvp-public-frontend-v2.md` *(reconciled against locked architecture, BA memo, and UX review). Do not implement from this file.*

Status: Not started **Preconditions:** Gates 05 and 06 complete (API endpoints live; real species data seeded) **Unlocks:** ECA Workshop demonstration — this is the MVP

Purpose

Build the public-facing Next.js frontend. This is the product that conservation stakeholders will see at the June 2026 ECA Workshop. Three pages: species directory, species profile, conservation dashboard.

No auth UI in the Next.js frontend at MVP. Coordinators use Django Admin. The Next.js app is a public-facing read-only interface over the DRF API.

Deliverables

- Next.js 14 project with TypeScript and Tailwind CSS
- Four public pages: `/species/`, `/species/[id]/`, `/dashboard/`, `/map/`
- Navigation header and footer present on all pages
- SSR/SSG for species directory and species profiles (for SEO on public species pages)
- Configurable API base URL via `NEXT_PUBLIC_API_URL` environment variable
- Deployed to staging at a public URL accessible to ECA Workshop attendees

---

API Dependencies (Gate 05)

Page	DRF Endpoints Consumed
Species Directory	`GET /api/v1/species/`
Species Profile	`GET /api/v1/species/{id}/`, `GET /api/v1/field-programs/` (for linked programs)
Conservation Dashboard	`GET /api/v1/dashboard/`
Conservation Map	`GET /api/v1/map/localities/`, `GET /api/v1/map/watersheds/`, `GET /api/v1/map/summary/`
Conservation Map (static)	`GET /static/map-layers/watersheds.geojson`, `GET /static/map-layers/protected-areas.geojson`

All requests are unauthenticated (Tier 1 / public access). No token management in the Next.js frontend at MVP.

User Stories

FE-07-1: Species Directory

As a conservation stakeholder visiting the platform, **I want** to browse all ~95–100 Madagascar freshwater fish species in a searchable, filterable directory, **so that** I can find species I'm interested in and understand the overall scope of the group.

Page: `/species/` **Rendering:** SSG with revalidation every 24 hours (species data changes rarely; IUCN sync is weekly)

UI elements:

- Page header: "X Endemic Freshwater Fish Species" with subtitle "(Y described, Z undescribed taxa)"
- Counts come from `count`, `described_count`, `undescribed_count` in the API response
- Filter sidebar or filter bar:
  - Taxonomic status (All / Described / Undescribed)
  - IUCN status (multi-select: CR / EN / VU / NT / LC / DD / NE)
  - Family (multi-select, populated from species data)
  - CARES status (All / CCR / CEN / CVU / CLC)
- Search input (submits to `?search=...` query param)
- Species cards/rows: scientific name, taxonomic status badge, family, IUCN status badge, CARES status badge
- Undescribed taxa display a "Provisional Name" badge distinguishing them from described species
- IUCN status badges use standard IUCN color coding (CR = red, EN = orange, VU = yellow, etc.)
- Pagination: 50 per page; page controls at bottom
- Filters update URL query params (shareable/bookmarkable filter state)

Acceptance Criteria:

Given a user visiting `/species/` **When** the page loads **Then** the header shows the correct total count; both described and undescribed species appear by default; pagination controls are visible if more than 50 species exist

Given a user with `?taxonomic_status=undescribed_morphospecies` in the URL **When** the page loads **Then** only undescribed taxa are shown; the count reflects the filtered set; each card shows the "Provisional Name" badge

Given a user selecting "CR" from the IUCN status filter **When** the filter is applied **Then** only CR-status species are shown; the count updates; the filter state is reflected in the URL

Given a user typing "sakaramy" in the search field **When** the search is submitted **Then** *Pachypanchax sakaramyi* appears in results; other species do not

Given a user clicking a species card **When** the click is registered **Then** the user is navigated to `/species/{id}/`

FE-07-2: Species Profile

As a conservation stakeholder or educator, **I want** to view a detailed profile page for a single species, **so that** I understand its taxonomy, conservation status, ecological context, and captive population status.

Page: `/species/[id]/` **Rendering:** SSR with `getServerSideProps` (individual species pages may update after IUCN sync; SSR ensures freshness without cache invalidation complexity at MVP)

UI sections:

1. **Header** — Scientific name (bold, italic), taxonomic status badge, family, genus. For undescribed taxa: "Undescribed taxon — provisional name used in conservation literature" notice below the name. Do not display authority or year_described for undescribed taxa.
2. **Conservation Status** — IUCN status badge (category + criteria string), assessment date, CARES status badge. No mention of `review_status` to public users (pending reviews are invisible at Tier 1).
3. **Common Names** — Displayed by language (English, French, Malagasy)
4. **Description & Ecology** — `description`, `ecology_notes`, `distribution_narrative`, `morphology`, `max_length_cm`, `habitat_type` — displayed if non-empty
5. **Captive Population Summary** — "X institutions hold this species in captivity, with approximately Y individuals across Z breeding programs." Uses `ex_situ_summary` from the API. If `institutions_holding = 0`: "No captive population is currently tracked for this species." Use warm neutral styling to convey urgency without alarming language.
6. **Field Programs** — List of linked field programs (name, status, region) with links to `/field-programs/{id}/` (if that page is implemented; otherwise link to program website if available). Can be empty section if no programs linked.
7. **External Links** — If `iucn_taxon_id` is set: link to IUCN Red List species page. If `gbif_taxon_key` is set: link to GBIF species page.

Acceptance Criteria:

Given a user visiting `/species/{id}/` for *Bedotia sp. 'manombo'* (undescribed morphospecies) **When** the page renders **Then** the header shows "Bedotia sp. 'manombo'" with a "Provisional Name" badge; the "Undescribed taxon" notice is displayed; no authority or year is shown; the IUCN status section shows "Not Evaluated" if no assessment exists

Given a user visiting `/species/{id}/` for *Pachypanchax sakaramyi* (described, IUCN EN) **When** the page renders **Then** the IUCN status badge shows "EN" in orange; the criteria string "B1ab(iii)" is displayed; the assessment date and assessor are shown; the authority "Holly, 1928" is shown

Given a user visiting `/species/{id}/` for a species with `ex_situ_summary.institutions_holding = 0` **When** the page renders **Then** the captive population section displays "No captive population is currently tracked for this species" — not an error state, not empty

Given a user visiting `/species/9999/` where no such species exists **When** the page renders **Then** a 404 page is displayed (Next.js `notFound()`)

Given the DRF API is unreachable during SSR **When** the page is requested **Then** an error page is shown with a message like "Data temporarily unavailable" — not a stack trace

---

FE-07-3: Conservation Dashboard

As a conservation funder, workshop attendee, or journalist, **I want** to see a visual summary of the ex-situ coverage gap and overall conservation status, **so that** I understand at a glance why this platform exists and what problem it solves.

Page: `/dashboard/` **Rendering:** SSR (dashboard data refreshes after IUCN sync; freshness matters here)

UI sections:

1. **The Coverage Gap** (most prominent section) — Large stat block: "X of Y threatened species have no known captive population." Threatened = IUCN CR + EN + VU. Uses `ex_situ_coverage.threatened_species_without_captive_population` and `ex_situ_coverage.threatened_species_total`.
2. **Species by IUCN Threat Status** — Horizontal bar chart or donut showing species counts by IUCN category. Use standard IUCN colors. Include "Not Evaluated / Not Assessed" category for species without assessments.
3. **Summary Stats Row** — Three stat boxes: total species tracked, institutions participating, total populations monitored. Uses `species_counts.total`, `ex_situ_coverage.institutions_active`, `ex_situ_coverage.total_populations_tracked`.
4. **Field Programs Active** — Simple count with brief description. Uses `field_programs.active`.
5. **Data freshness** — Small footer note: "IUCN assessment data last synced: [date]" using `last_updated`.

Chart library: Use `recharts` or `chart.js` via `react-chartjs-2` (both are lightweight, React-compatible, and open-source). No proprietary visualization dependencies.

Acceptance Criteria:

Given a user visiting `/dashboard/` **When** the page loads **Then** the coverage gap stat prominently shows "31 of 49 threatened species have no known captive population" (or whatever the current data shows); the species-by-status chart renders with data for all present categories

Given `GET /api/v1/dashboard/` returns `last_updated: "2026-04-10T02:00:00Z"` **When** the dashboard renders **Then** the data freshness note reads "IUCN data last synced: April 10, 2026" (formatted, not raw ISO timestamp)

Given a user on a mobile device (viewport < 768px) **When** the dashboard renders **Then** the layout stacks vertically; charts are responsive; no horizontal scroll is required; text remains readable at default font size

---

FE-07-4: Navigation and Site Shell

As any visitor, **I want** consistent navigation across all pages, **so that** I can move between the directory, profiles, and dashboard without confusion.

Header links: "Species Directory" (`/species/`), "Map" (`/map/`), "Conservation Dashboard" (`/dashboard/`), "About" (static page or anchor on homepage — minimal content at MVP)

This follows the logical flow: browse species (directory) → see where they are (map) → understand the crisis (dashboard) → learn about the platform (about).

Footer: Platform name, Apache-2.0 license notice, link to GitHub repository, "Data sources: IUCN Red List, Leiss et al. 2022"

Acceptance Criteria:

Given a user on any page **When** they click "Species Directory" in the navigation **Then** they are taken to `/species/` and the current page link in the nav is visually active

Given a user on any page **When** the page is viewed without JavaScript **Then** the species directory and profile pages still render meaningful content (SSR/SSG guarantees this; dashboard chart may degrade gracefully)

FE-07-5: Conservation Map Page

As a workshop attendee, conservation professional, or public visitor, **I want** to explore an interactive map of Madagascar showing where freshwater fish species have been recorded, **so that** I can understand the geographic distribution of this species group and how it relates to watersheds and protected areas.

Page: `/map/` **Rendering:** CSR (client-side rendering). Map interactions are inherently client-side; SSR provides no benefit.

Frontend dependencies:

- `leaflet` + `react-leaflet` (React wrapper for Leaflet)
- `leaflet.markercluster` (marker clustering plugin)
- No additional mapping libraries

This story is structured in two tiers. Tier A is must-ship; Tier B ships if time allows.

Tier A: Must-Ship (estimated ~1 week frontend effort)

UI elements:

1. **Full-viewport Leaflet map** centered on Madagascar (approximately -18.9, 47.5, zoom 6)

2. **Base layer switcher** — OpenStreetMap (default) and ESRI World Imagery (satellite) as switchable base layers via Leaflet's built-in layer control

3. **Species locality point markers:**

   - Fetched from `GET /api/v1/map/localities/`
   - Color-coded by IUCN status using standard IUCN colors: CR=red, EN=orange, VU=yellow, NT=near-green, LC=green, DD=gray, NE=light gray
   - Shape/style varies by locality type: type_locality uses a star or diamond shape; other types use standard circle markers
   - Fill style varies by presence status: solid fill = present; hollow (outline-only) = extirpated; dashed outline = unknown; distinct marker = reintroduced
   - Marker clustering via `leaflet.markercluster` for decluttering at low zoom levels

4. **Marker popups** — click a locality marker to see:

   - Scientific name (italic, linked to `/species/{id}/`)
   - IUCN status badge
   - Locality name
   - Locality type (e.g., "Type Locality")
   - Presence status
   - Water body name
   - Year collected (if available)
   - Source citation
   - Coordinate precision indicator (e.g., "Location precision: exact")
   - For sensitive records with generalized coordinates: "Location generalized to protect sensitive species"

5. **Legend:**

- Color key for IUCN status (CR, EN, VU, NT, LC, DD, NE with corresponding colors)
- Shape key for locality type (type_locality visually distinct from others)
- Fill/outline key for presence status (solid=present, hollow=extirpated, dashed=unknown, distinct for reintroduced)

6. **"View on Map" cross-link from species profiles** — on `/species/[id]/` pages, add a "View on Map" button that links to `/map/?species_id={id}`. Button is only shown when the species has at least one locality record.

Tier A Acceptance Criteria:

Given a user visiting `/map/` **When** the page loads **Then** a Leaflet map renders centered on Madagascar with OpenStreetMap tiles; species locality point markers are visible with IUCN color coding; the legend is visible

Given a user visiting `/map/` and the localities API returns zero records **When** the page loads **Then** the map renders with the base map and no markers; a message reads "Locality data is being compiled — check back soon"; no error state is displayed

Given a user visiting `/map/` and the localities API is unreachable **When** the page loads **Then** the base map still renders (tiles are from CDN); an inline error message indicates "Species locality data temporarily unavailable"

Given a user clicks on a species locality marker **When** the popup opens **Then** all specified popup fields are displayed; the scientific name is a clickable link to the species profile; coordinate precision is shown in the popup text

Given multiple markers overlap at the current zoom level **When** the user clicks the cluster **Then** the map zooms in to reveal individual markers (standard markercluster behavior)

Given a user clicks the species name link in a marker popup **When** the navigation occurs **Then** the user arrives at `/species/{id}/` for that species

Given a user switches from OpenStreetMap to ESRI satellite base layer **When** the switch occurs **Then** the base tiles change; all point markers remain visible and correctly positioned

Given a user viewing a species profile at `/species/{id}/` for a species with locality records **When** the page renders **Then** a "View on Map" button is visible

Given a user clicks "View on Map" on the profile for *Pachypanchax sakaramyi* **When** the map page loads **Then** the URL is `/map/?species_id={id}`; the map shows only that species' localities; the map auto-zooms to fit the extent of that species' points

Given a user viewing a species profile for a species with ZERO locality records **When** the page renders **Then** the "View on Map" button is absent or disabled with tooltip "No locality data available"

Given a user arrives at `/map/?species_id=9999` (nonexistent species) **When** the map page loads **Then** the map renders with all localities (no filter); an inline notice reads "Species not found — showing all localities"

Given a user on a tablet (768-1024px viewport) **When** the map page loads **Then** the map fills the available viewport; touch gestures (pinch zoom, pan) work correctly; marker popups fit within the viewport

Given a user on a mobile phone (< 768px viewport) **When** the map page loads **Then** the map renders at full width and at least 60% viewport height; the legend is collapsible or minimized

Tier B: Ship If Time Allows (estimated ~1-2 weeks additional frontend effort)

Tier B features build on Tier A. None of Tier B should be started until Tier A is complete and tested.

UI elements:

7. **Watershed polygon overlay** (toggleable, off by default):
    - Loaded lazily from `/static/map-layers/watersheds.geojson` on first toggle
    - Semi-transparent styling; does not obscure point markers beneath
    - Loading indicator while GeoJSON is fetching
    - Click a watershed polygon → popup/sidebar shows: watershed name, area in km2, list of species in that basin (from already-loaded locality data, filtered client-side by `drainage_basin_name`), each species name linked to profile with IUCN badge
8. **Protected area polygon overlay** (toggleable, off by default):
    - Loaded lazily from `/static/map-layers/protected-areas.geojson` on first toggle
    - Visually distinct from watershed overlay (different color, opacity, border style)
    - Loading indicator while GeoJSON is fetching
    - Point markers remain clickable through polygon overlays
    - Locality markers render above both polygon layers in z-order
9. **Filter panel** (collapsible sidebar):
    - Family dropdown (populated from locality data or species list)
    - IUCN status multi-select
    - Locality type multi-select
    - Presence status multi-select
    - Watershed dropdown (populated from `/api/v1/map/watersheds/`)
    - Coordinate precision multi-select
    - Filters update markers via re-request to `/api/v1/map/localities/` with filter params
    - Filter state reflected in URL query parameters (shareable links)
    - On mobile/tablet: filter panel accessible via button/drawer, not permanently visible
10. **Map statistics bar** (top or bottom of map):
    - "X localities for Y species" (updates with active filters)
    - Data from `/api/v1/map/summary/` for initial load; updated client-side when filters are applied

Tier B Acceptance Criteria:

Given a user toggles the "Watersheds" layer on for the first time **When** the static GeoJSON is loading **Then** a loading indicator appears on the layer toggle; the map remains interactive; once loaded the polygons render

Given a user clicks on a watershed polygon (e.g., Betsiboka) **When** the popup renders **Then** it shows the watershed name, area, and a list of species with localities in that basin; each species name links to its profile; if no species exist: "No species locality records in this basin"

Given a user clicks on an unnamed watershed **When** the popup renders **Then** the name displays as "Sub-basin of [parent name]" or "Basin [Pfafstetter code]" — never blank, never "null"

Given both "Watersheds" and "Protected Areas" layers are toggled on **When** both layers render **Then** they use distinct visual styling (different colors, opacity, borders); point markers render above both polygon

layers

Given a user on a slow connection (simulated 3G) toggles the protected areas layer **When** loading is in progress **Then** a loading state is visible; the map does not freeze; the user can cancel by toggling the layer off

Given a user selects "CR" in the IUCN status filter **When** the filter is applied **Then** only CR-status localities are visible; the statistics bar updates; the URL reflects the filter state

Given a user filters by `presence_status = "historically_present_extirpated"` **When** the filter is applied **Then** only extirpated localities are shown; the statistics bar updates

Given a user on a tablet with the filter panel open **When** they select a family filter **Then** the filter applies immediately; the map updates; the filter panel remains open for additional selections

Given a user on mobile (< 768px) **When** the map page loads **Then** the filter panel is hidden by default; accessible via a button/drawer; statistics bar collapses to essential info

---

Technical Tasks

- Initialize Next.js 14 project with TypeScript: `npx create-next-app@14 --typescript --tailwind --app`
- Use the App Router (`app/` directory) for all pages
- Create TypeScript types for all DRF API response shapes in `lib/types.ts` ; these are the client-side reflection of the DRF serializer contracts
- Create `lib/api.ts` — typed fetch wrapper that reads `NEXT_PUBLIC_API_URL` from env; handles API errors consistently
- Configure `next.config.js` to proxy `/api/` to the Django backend in development (avoids CORS issues)
- Install and configure chart library (recharts recommended)
- Install `leaflet` , `react-leaflet` , and `leaflet.markercluster` for the map page
- Create map page components: `MapView` , `LocalityMarker` , `MapLegend` , `LayerToggle` , `FilterPanel` (Tier B)
- Add `.env.local.example` with `NEXT_PUBLIC_API_URL=http://localhost:8000`
- Write Vitest component tests for: species card renders undescribed taxa badge correctly, dashboard coverage gap stat renders correct values, IUCN status badge applies correct color class
- Deploy to Fly.io or Vercel staging environment before gate is marked complete

---

Out of Scope

- Login, registration, or any auth UI (coordinators use Django Admin)
- Species profile pages for occurrence data, survey maps, or breeding records (post-MVP)
- Coordinator-facing Next.js UI (Gate 08)
- Field program data entry (post-MVP)
- Multilingual UI (English only at MVP; CommonName data for fr/mg is stored but not surfaced in UI)
- Species images (stored in S3; display is desirable but not required for ECA Workshop demonstration)
- Map-based data entry or user-contributed locality submissions via frontend (post-MVP; use Django Admin)
- IUCN range polygons on the map (different data source; post-MVP)
- Survey gap analysis or habitat suitability overlays (post-MVP)
- Species profile embedded mini-maps (deferred; "View on Map" link provides the same navigation)

- Dashboard map widget (deferred; dashboard tells a statistical story, map tells a geographic story)

Gate Exit Criteria

This is the MVP gate. All prior gates must be complete before this one is marked complete.

Before marking Gate 07 (MVP) complete:

1. All four pages render with real seeded data (`/species/` , `/species/{id}/` , `/dashboard/` , `/map/`)
2. The species directory correctly shows described vs. undescribed species with appropriate badges
3. The dashboard coverage gap statistic renders correctly
4. The map page renders with seeded locality data; markers are color-coded by IUCN status; legend is visible and accurate
5. Marker popups display all specified fields including coordinate precision
6. "View on Map" link appears on species profiles with localities; absent for species without localities
7. Map page is responsive on tablet (768-1024px) and mobile (< 768px) viewports
8. Navigation header includes "Map" link in the correct position (Species Directory, Map, Conservation Dashboard, About)
9. Empty state (zero localities) displays graceful message, not an error
10. API failure state on map shows base map with inline error message
11. No stack traces or unhandled errors appear on any page with valid data
12. Pages render on mobile viewports without horizontal scroll
13. Deployed to a staging URL that can be demonstrated at the ECA Workshop (June 1–5, 2026)
14. **If Tier B ships:** filter panel works; watershed/PA overlays load lazily; statistics bar updates with filters
15. Invoke **@test-writer** to write adversarial frontend tests (undefined API fields, empty states, network failure, 404 species IDs, empty locality response, malformed GeoJSON, species_id filter with nonexistent ID, slow-loading reference layers)
16. Invoke **@ux-reviewer** — this is the face of the platform at the workshop; validate that the coverage gap message, species directory, and conservation map are immediately comprehensible to a conservation professional unfamiliar with the platform; verify map legend is comprehensible, marker colors distinguishable, popup information hierarchy clear, and mobile layout works for workshop tablet demos
17. Invoke **@code-quality-reviewer** on Next.js code
18. Invoke **@security-reviewer** — verify no sensitive data leaks through public API calls; verify NEXT_PUBLIC env vars contain nothing sensitive